

Software Requirements Specification

Project: EventSure Suite

Domain: InsurTech & FinTech Automation

Backend: .NET 9.0 Clean Architecture

Frontend: Angular 21 Signals-Driven SPA

Project Lead: [Your Name/ID]

Submission Date: March 12, 2026

Status: Final Technical Draft

Contents

1	1. Introduction	2
1.1	1.1 Purpose	2
1.2	1.2 Document Conventions	2
1.3	1.3 System Overview	2
1.4	1.4 Technology Stack	2
2	2. Functional Requirements	3
2.1	2.1 [FR-01] Identity Access Management	3
2.2	2.2 [FR-02] Heuristic Risk Premium Engine	3
2.3	2.3 [FR-03] Interactive InsureCalendar	3
2.4	2.4 [FR-04] Claims Assessment Audit	3
3	3. System Architecture & Data Design	4
3.1	3.1 The Clean Architecture Concept	4
3.2	3.2 Entity Relationship Mapping (Core)	4
3.3	3.3 API Endpoint Dictionary	4
4	4. Non-Functional Requirements	5
4.1	4.1 Security Specifications	5
4.2	4.2 UI Aesthetics (EventSure Design System)	5
4.3	4.3 Reliability & Performance	5
5	5. Milestones & Roadmap	6
5.1	5.1 Development Phases	6
5.2	5.2 Component Responsibility Matrix	6

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed lifecycle requirements specification for the **EventSure Suite**. This application serves as a comprehensive middleware bridging Event Organizers and Insurance Underwriters through automated risk assessment.

1.2 Document Conventions

This document follows the IEEE-830-1998 standards for SRS documentation. All functional requirements are tagged with [FR-XX] for traceability.

1.3 System Overview

EventSure is a multi-tier enterprise application. Unlike traditional insurance portals, EventSure implements a **Heuristic Logic Engine** to analyze event telemetry (Alcohol presence, Pyro usage, Crowd density) to adjust premiums programmatically.

1.4 Technology Stack

- **Frontend:** Angular 21, RxJS 7, Tailwind 4 (Signal Architecture).
- **Backend:** ASP.NET Core 9, C# 13, Entity Framework Core.
- **Database:** Microsoft SQL Server (Transact-SQL).
- **Auth:** Stateless JWT Bearer with RBAC (Role-Based Access Control).

2. Functional Requirements

2.1 [FR-01] Identity Access Management

- **Description:** Secure authentication for four roles: Customer, Agent, Admin, Claims Officer.
- **Logic:** The system shall utilize JWT tokens with 1-hour expiry and encrypted Role claims.

2.2 [FR-02] Heuristic Risk Premium Engine

Core Logic Requirement: The system shall calculate the risk-adjusted premium using a Heuristic Evaluating Function. **The Heuristic Equation:**

$$P_{final} = (C \times R_{base}) \times (1 + \omega_{attend} + \omega_{venue} + \omega_{hazard}) \quad (1)$$

Where:

- ω_{attend} : +0.30 if attendees > 5,000.
- ω_{venue} : +0.10 if Outdoor (Weather Risk).
- ω_{hazard} : +0.25 if Fireworks are utilized.

2.3 [FR-03] Interactive InsureCalendar

- **Description:** A reactive monthly matrix view for Customers.
- **Interactivity:** Markers for *PolicyStart* (Emerald), *PolicyEnd* (Rose), and *PremiumDue* (Yellow Pulse).

2.4 [FR-04] Claims Assessment Audit

- **Description:** Claims Officers must access an audit trail of policy payment history.
- **Integrity:** Claim cannot be approved if > 20% of premiums are unpaid unless authorized by Admin.

3. System Architecture & Data Design

3.1 The Clean Architecture Concept

The backend is structured into four distinct layers to minimize dependency coupling:

1. **Web.API:** Entry point, Middleware, Controllers.
2. **Application:** Interfaces, Services, DTO mapping, and `PremiumCalculator`.
3. **Domain:** POCO Entities, Enum status types, Custom Exceptions.
4. **Infrastructure:** `DbContext`, Migrations, Repositories, and JWT Generation.

3.2 Entity Relationship Mapping (Core)

Table	Key	Attributes	Relationship
Users	UserID	RoleID, Email, PasswordHash	1:M with Requests
ActivePolicy	PolicyID	TotalPremium, ExpiryDate	1:M with Payments
Claims	ClaimID	Reason, Amount, StatusBit	M:1 with Policy

Table 1: Relational Database Schema (Simplified)

3.3 API Endpoint Dictionary

- `POST /api/auth/login` - Returns token and identity payload.
- `GET /api/customer/calendar` - Fetches policy dates for the active matrix.
- `PATCH /api/notifications/read-all` - Bulk status update for UI sync.

4. Non-Functional Requirements

4.1 Security Specifications

1. **CORS Policy:** Strictly allows *localhost:4200*.
2. **Password Strategy:** PBKDF2 with Salt (implemented via ASP.NET Identity).
3. **Injection Prevention:** All raw SQL is prohibited; EF Core LINQ parameterized queries are mandatory.

4.2 UI Aesthetics (EventSure Design System)

The user interface follows the **Glassmorphism** design principle.

- **Background:** Semi-transparent `bg-white/70` with `backdrop-blur-xl`.
- **Color Palette:** Dynamic Blue (`#3b82f6`) and Vibrant Yellow (`#eab308`).
- **Animations:** 60fps framerate for *shimmer* and *float* keyframes to ensure premium responsiveness.

4.3 Reliability & Performance

- **Uptime:** Target 99.9% via Global Exception Handling middleware.
- **Response Time:** Heuristic calculations shall complete in $< 150ms$.

5. Milestones & Roadmap

5.1 Development Phases

Phase 1 : Infrastructure setup (EF Core Migrations, DB Seeding).

Phase 2 : Core Engine Implementation (Premium Heuristics Auth Flow).

Phase 3 : Portal Componentization (Customer, Agent, Claims Dashboards).

Phase 4 : Visual Polish (Tailwind 4 Animations, InsureCalendar, Ads).

Phase 5 : Stability Audit (Unit Testing, API Log Analysis).

5.2 Component Responsibility Matrix

Task / Role	Cust	Agent	Admin	CO
Request Insurance	X			
Verify Payment		X		
Manage Products			X	
Audit Claim				X

Table 2: Functional Permission Matrix

— End of Document —